

Caching and Database Scaling Without Stale Promises

A cache stores a reusable result under an eligibility and freshness policy. It is not automatically an authoritative source. Replicas and partitions address different scaling needs, and every optimization changes the failure path. Start with measured access patterns and explicit staleness tolerance.

For: Backend engineers who want to use caches, replicas, and partitions deliberately while preserving correctness and avoiding cache stampedes or misleading consistency claims.

Suggested pace: 3 weeks with a local service | Database and API basics

What you will be able to do

- Choose cache keys and freshness rules that respect identity and permissions.
- Explain cache-aside, invalidation, expiration, and stampede protection.
- Distinguish read replication from write partitioning and handle hot keys.
- Plan recovery when a cache or replica becomes unavailable.

Build the right mental model

Cache correctness starts with the key

Include every input that affects the result, such as tenant, user scope, query, version, or locale where relevant. Similar requests are not necessarily interchangeable. Private responses must not become shared merely because their URL looks the same.

Freshness is an explicit trade-off

TTL bounds some stale periods but does not eliminate race conditions or guarantee immediate revocation. Invalidations can be delayed or lost. For critical permissions or financial state, perform an authoritative check at the decision boundary.

Scaling methods solve different problems

Read replicas can increase read capacity but may lag and do not generally multiply a primary's write capacity. Sharding distributes ownership and can complicate cross-shard transactions, rebalancing, and queries. A hot key may remain hot regardless of shard count.

Study with evidence.

Add a cache only after measuring a repeatable expensive read. Document the source of truth, cache key, expiration, invalidation, and fallback. Test cache loss and stale data, not just the warm-cache happy path.

Week 1: Cache contracts

Implement cache-aside for a safe read and measure hit rate, miss latency, and backend load. Define positive and negative caching separately. Check user and tenant scope and ensure expiry does not change an authorization decision.

Deliverable and check: A cache-key specification and a test showing two users cannot receive each other's private result.

Week 2, first half: Stampedes and invalidation

Simulate many misses for one key and add request coalescing or bounded refresh. Consider jittered expiry and stale serving only where allowed. Trace a write racing with a read refill.

Deliverable and check: A failure test demonstrating bounded backend load and a documented stale-data policy.

Week 2, second half: Replicas and partitions

Compare read replicas, vertical changes, and sharding against the actual workload. Describe lag, routing, hot partitions, and transaction boundaries. Avoid introducing multiple storage systems without an ownership reason.

Deliverable and check: A scaling decision note with rejected alternatives and consistency consequences.

Week 3: Recovery and measurement

Remove the cache or delay a replica in a local test. Apply backpressure, admission limits, and safe fallbacks. Measure p95/p99 and error rates alongside hit ratio, then document recovery behavior.

Deliverable and check: An operational plan that protects the database when the optimization layer disappears.

A repeatable practice session

Spend about 15 minutes recalling the previous topic without notes, 45 minutes on today's problem or lab, and 15 minutes explaining and checking the result. Record the failed assumption, not just the final answer. Revisit weak material after a few days and again the following week.

Apply it. Explain it.

Prevent a hot-key cache stampede

A popular product page expires from cache and hundreds of requests arrive together. Each independently queries the database, creating a burst much larger than the ordinary miss rate suggests.

A worked approach

1. Measure concurrent misses for the same key and the database query cost. Distinguish one hot key from a general capacity problem. Confirm whether serving a slightly stale product description is acceptable; price or permission checks may need a different contract.
2. Coalesce refresh work so one request or worker refreshes a key while others wait within a deadline or receive allowed stale content. Bound waiters and refresh duration, and ensure a failed refresher does not leave a permanent lock.
3. Add expiry jitter across keys and backend admission limits. Test a failed refresh, cache outage, and simultaneous expiry. Verify that recovery does not overwhelm the database with all queued refreshes at once.

Expected result

The cache reduces repeated work while the miss path remains controlled. The design states which data may be stale and does not use cached product information as authoritative checkout validation.

Verify the result

- Is refresh ownership released after timeout or failure?
- Can the database survive a cold-cache start?
- Are private and critical decisions excluded from unsafe stale serving?

Extend the exercise

Change the hot entity from a product description to a revoked private document. Explain why eventual cache invalidation alone is insufficient and where an authoritative permission check belongs.

Make the learning stick.

Does a high hit ratio prove the cache is successful?

Not alone. Miss cost, tail latency, memory, backend protection, and correctness matter. A cache can have an excellent average hit ratio while a few hot-key misses cause severe overload.

Does adding replicas solve write scaling?

Usually not for a single-primary write path. Replicas copy work and serve reads under their consistency model. Write throughput may require changing data ownership, batching, schema, or partitioning, with additional trade-offs.

Can cache invalidation be made irrelevant with a short TTL?

No. Short TTLs trade more misses for a smaller typical stale window, but critical revocations and races still need explicit handling. The acceptable freshness policy comes from the business contract.

Avoid these traps

- Caching private results under a shared incomplete key.
- Ignoring the cold-cache and cache-outage load on the database.
- Treating replicas and shards as interchangeable scaling tools.

Ready for the next step?

- Source of truth and cache eligibility are explicit.
- Misses and refreshes are bounded under concurrency.
- Scaling choices preserve the required consistency contract.
- Outage recovery has been tested rather than assumed.

Reliable next reads

- [Azure cache-aside pattern](#)
- [MDN HTTP caching](#)
- [Azure sharding pattern](#)

An independent preparation guide, not an employer-endorsed syllabus or a guarantee of an offer. Company examples are practice contexts, not verified past interview questions. Use only public, synthetic, or authorized material in exercises.